TruffleHog Vorbereitung

Für Workshop-Verantwortliche | Materialien, Testdaten und Abnahme

Rahmen

Einzelarbeit; 100 Minuten reine Aufgabenzeit. Zusätzlich etwa 10 Minuten Einführung und 10 Minuten Auswertung einplanen. Installation und Funktionsprüfung vor dem Workshop erledigen.

Vorab bereitstellen

1. Eine festgelegte, praktisch geprüfte TruffleHog-Version sowie Git und einen Texteditor. TruffleHog muss über den Befehl trufflehog erreichbar sein.

2. Git Bash für Windows oder eine Shell unter macOS/Linux. Gemeinsame Befehle in genau diesen Umgebungen prüfen.

3. Das Teilnehmer-PDF, das vollständige Workshop-Paket und eine getrennte Musterlösung.

4. Eine kommentierte Konfigurationsvorlage und eine kurze Erklärung von Präfix, Zeichenklasse, Wiederholungsanzahl und Wortgrenze.

5. Optional vorbereitete Ergebnisbeispiele für Personen mit technischen Problemen; ausdrücklich als Beispiele kennzeichnen.

Struktur des auszuliefernden Pakets

```
workshop/
  workshop.yml
  aufgabe1/app/
  aufgabe2/app/
  aufgabe6/
    .git/
    app/
  aufgabe7/
    meine-regel.yml
    app/
  aufgabe8/
    .git/
    erweiterte-regeln.yml
    app/
```

Aufgaben 3 bis 5 verwenden aufgabe2/app. Aufgabenblätter, Lösungen und Scan-Ausgaben liegen außerhalb der app-Ordner. Historienübungen enthalten eine vollständige .git-Historie; ein gewöhnliches GitHub-Quellcode-ZIP genügt nicht.

Testdaten und Soll-Ergebnisse

Aufgaben 1 bis 6

Gemeinsame Erkennungsregel: workshop.yml

```
detectors:
  - name: hhn-workshop-secret
    keywords:
      - HHN-DEMO-SECRET-
    regex:
      secret: 'HHN-DEMO-SECRET-[A-F0-9]{8}\b'
```

Keinen verify-Abschnitt ergänzen. Mit --no-verification laufen die Übungen ohne aktive Prüfung der Credentials. --no-update unterbindet Update-Prüfungen.

Aufgabe 1: Ein eindeutiger Treffer

app/settings.env mit API_KEY=HHN-DEMO-SECRET-7F2A91BC. Zusätzlich eine unkritische Datei, z. B. app.py. Soll: Marker in settings.env wird erkannt.

Aufgabe 2: Mehrere Fundorte

Datei unter aufgabe2/app	Vorbereiteter Inhalt
.env	API_KEY=HHN-DEMO-SECRET-7F2A91BC
config.json	{"api_key":"HHN-DEMO-SECRET-A83D60E2"}
backup/old-config.txt	TOKEN=HHN-DEMO-SECRET-19CE42AF
notes.txt	Passwort: password123
app.py	print("Workshop-Anwendung")

Soll: drei unterschiedliche Workshop-Marker in drei Dateien; password123 wird von der eigenen Regel nicht erkannt. Nicht pauschal die exakte Zahl der ausgegebenen Ergebniszeilen bewerten.

Aufgaben 3 bis 5: Ergebnisse erklären

Einen echten Beispiel-Fund der gewählten Version als kommentierte Terminal- und JSON-Ausgabe bereitstellen. Dateipfad, Detektor, Raw-Wert und Verifikationsangaben markieren. Soll: unverified-Funde sichtbar; bei --no-verification und --results=verified keine bestätigten Workshop-Funde.

Aufgabe 6: Vorbereitete Git-Historie

Drei Commits erstellen: A unkritisches Grundprojekt; B Marker in app/settings.env hinzufügen; C Marker durch einen unkritischen Platzhalter ersetzen. Aktuelle app-Dateien dürfen keinen Marker enthalten. Soll: Dateiscan ohne Workshop-Fund; Git-Scan findet den historischen Marker. Kennungen von B und C in der Musterlösung festhalten.

Eigene Regeln und Abschlussprojekt

Aufgaben 7 und 8

Aufgabe 7: Kommentierte Vorlage

meine-regel.yml mit klar markierten Platzhaltern für Präfix und Ziffernanzahl ausgeben. Die unvollständige Vorlage muss vor dem Scan ergänzt werden. Zusätzlich eine vollständig ausgefüllte Lösung getrennt bereithalten.

```
detectors:
  - name: campus-token
    keywords:
      - CAMPUS-TOKEN-
    regex:
      token: 'CAMPUS-TOKEN-[0-9]{4}\b'
```

Kurzhilfe: [0-9] steht für eine Ziffer; {4} für genau vier Wiederholungen; \b für eine Wortgrenze. Nach der Änderung lautet die Wiederholungsanzahl {6}. Die Wortgrenze verhindert hier Teiltreffer in längeren Ziffernfolgen.

Testwert	Regel: 4 Ziffern	Regel: 6 Ziffern
CAMPUS-TOKEN-1234	Treffer	kein Treffer
CAMPUS-TOKEN-654321	kein Treffer	Treffer
CAMPUS-TOKEN-123	kein Treffer	kein Treffer
CAMPUS-TOKEN-12345	kein Treffer	kein Treffer
CAMPUS-TOKEN-12A4	kein Treffer	kein Treffer
OTHER-TOKEN-1234	kein Treffer	kein Treffer

Aufgabe 8: Kombiniertes Testrepository

Ein neues Repository mit eigenständiger Historie erstellen. Aktuell enthält app/settings.env einen HHN-Marker, app/service.conf einen CAMPUS-Token mit sechs Ziffern und app/notes.txt unkritische Notizen. Ein anderer HHN-Marker steht ausschließlich in einem früheren Commit.

erweiterte-regeln.yml enthält beide Detektoren: HHN-Regel aus workshop.yml und CAMPUS-Regel mit sechs Ziffern. Die Datei liegt außerhalb von app. Die Teilnehmer führen zuerst den bekannten Scan und dann den Scan mit den zusätzlichen Regeln aus.

Soll: Dateiscan erkennt aktuelle Marker entsprechend der eingesetzten Konfiguration. Git-Scan findet zusätzlich den ausschließlich historischen HHN-Marker. Alle künstlichen Marker bleiben ohne bestätigte Gültigkeit. Musterlösung nach aktuellem und historischem Fund unterscheiden, nicht nach der absoluten Ergebniszeilenzahl.

Musterlösungen für alle Aufgaben

Pro Aufgabe: geprüfter Befehl, erwartete Fundorte, Beispielausgabe und kurze Antworten auf die Reflexionsfragen. Bei historischen Funden außerdem die tatsächlichen Commit-Kennungen aufnehmen. Keine Musterlösung im zu scannenden app-Verzeichnis speichern.

Durchführung und Abnahme

Vor der Ausgabe an die Teilnehmenden prüfen

Technischer Probedurchlauf

1. Alle Befehle aus Seite 5 des Teilnehmerblatts unverändert aus dem Paket-Wurzelverzeichnis ausführen. Pfade mit Leerzeichen und die verwendeten Windows-/macOS-/Linux-Umgebungen prüfen.

2. Aufgabe 1: Marker wird erkannt. Aufgabe 2: alle drei vorgesehenen Fundorte einschließlich Backup werden erkannt.

3. Aufgabe 3/4: Statusbeschreibungen und JSON-Beispiel stimmen mit der gewählten Version überein. Unverified und Unknown lassen sich nicht allein aus Verified: false unterscheiden.

4. Aufgabe 5: Filter liefern die vorgesehenen unterschiedlichen Ergebnisse. Es werden keine echten gültigen Credentials benötigt.

5. Aufgabe 6: aktueller Dateiscan ohne Workshop-Marker, historischer Scan mit Fund. Die übergebene Kopie enthält sämtliche benötigten Commits.

6. Aufgabe 7: beide vollständig ausgefüllten Regeln gegen alle sechs Testfälle prüfen. Die Musterlösung findet keine Teiltreffer in den längeren Testwerten.

7. Aufgabe 8: aktuelle und ausschließlich historische Fundorte prüfen. Erweiterte Konfiguration erkennt beide Formate.

8. Alle Teilnehmenden können das Paket lokal öffnen und TruffleHog starten. Installation und Paketverteilung sind abgeschlossen, bevor die Aufgabenzeit beginnt.

Hinweise für die Betreuung

Den ersten Scan gemeinsam vormachen, danach Einzelarbeit. Bei Schwierigkeiten zuerst Arbeitsverzeichnis, Dateipfad und Konfigurationspfad prüfen. Keine aktiven Credential-Prüfungen verlangen. Das Anpassungsprinzip bei Aufgabe 7 kurz erklären; reguläre Ausdrücke sind keine vorausgesetzte Kenntnis.

Wichtige Aussagen für die Auswertung

Erkennung bestätigt keine Gültigkeit. Unverified bedeutet nicht automatisch ungültig. Eine leere gefilterte Ausgabe beweist keine Secret-Freiheit. Löschen im aktuellen Stand bereinigt keine Historie. Bei einem echten veröffentlichten Schlüssel zuerst Widerruf/Austausch und möglichen Missbrauch behandeln.

Quellen zur technischen Vorbereitung

github.com/trufflesecurity/trufflehog
README: Custom Regex Detector
examples/generic.yml